Manual de deploy y configuración — Laqq

Fuente de verdad: `docker-compose.*.yml`, `scripts/`, `env.example`, `Backend/config/settings.py`, `Backend/entrypoint*.sh`, `.github/workflows/deploy.yml`.
Supersede parcial: consolida y actualiza lo descripto en `Backend/docs/DOCKER.md`, `Backend/docs/DEPLOY.md` y el README raíz cuando difieren del código actual.

---

1. Requisitos previos

Requisito	Notas
Docker + Docker Compose **v2**	Obligatorio para el flujo documentado
Git	Clonado del monorepo
Puertos libres	**Dev:** `3000` (FE), `8000` (API), `5433`→Postgres. **Prod:** `80`, `443`
Archivo `.env`	Copiar desde `env.example` en la raíz del monorepo
DNS / SSL (prod)	Dominios apuntando al servidor; certificados Let's Encrypt montados en el host (ver §4)

Opcional fuera de Docker: Node 20+ y Python 3.11+ (el CI usa Python 3.13 / Node 20).

---

2. Levantar el proyecto

2.1 Desarrollo

```
cp env.example .env
# Ajustar VITE_API_BASE_URL, DB_*, Turnstile de prueba, etc.

./scripts/dev-up.sh
# equivalente: docker compose -f docker-compose.dev.yml up --build
```

Servicio	Contenedor	URL / puerto
Frontend (Vite HMR)	`laqq-frontend-dev`	http://localhost:3000 (o `FRONTEND_PORT`)
Backend (runserver)	`laqq-backend-dev`	http://localhost:8000
PostgreSQL	`laqq-db-dev`	host `localhost:5433` → container `5432`

- Compose: `docker-compose.dev.yml`
- `LOAD_SEED_DATA` forzado a `"true"` en el servicio backend de dev
- Volúmenes: código montado en vivo (`./Backend`, `./Frontend`)

Apagar / logs:

```
./scripts/dev-down.sh
./scripts/dev-logs.sh
```

2.2 Producción (Docker)

1. Completar `.env` con valores reales (`DEBUG=False`, `SECRET_KEY` fuerte, `ALLOWED_HOSTS`, SMTP/Resend, Turnstile productivos, etc.).
2. Asegurar certificados y dir ACME en el host (ver §4).
3. Levantar:

```
./scripts/prod-up.sh
# equivalente: docker compose -f docker-compose.prod.yml up -d --build
```

Servicio	Contenedor	Exposición
Frontend + Nginx	`laqq-frontend`	`:80`, `:443`
Backend (Gunicorn)	`laqq-backend`	solo red interna
PostgreSQL	`laqq-db`	solo red interna

- En build del frontend, `VITE_API_BASE_URL` se fija a `/api` (ruta relativa vía Nginx).
- Nginx hace proxy `location ^~ /api/ → http://backend:8000/` (se **elimina** el prefijo `/api`).

Apagar / logs:

```
./scripts/prod-down.sh
./scripts/prod-logs.sh
```

---

3. Variables de entorno

Fuentes: `env.example`, lectura en `Backend/config/settings.py`, args de build en `docker-compose.prod.yml`.

3.1 Core Django / DB

Variable	Obligatoria	Propósito
`SECRET_KEY`	Sí (prod)	Firma crypto Django/JWT
`DEBUG`	Sí	`False` en prod
`DJANGO_ENV`	Compose	`development` / `production` (entrypoint: collectstatic si production)
`ALLOWED_HOSTS`	Sí	Hosts CSV permitidos

CSRF_TRUSTED_ORIGINS	Recomendada	Origins CSV; si se omite, defaults históricos de prod en settings
CORS_ALLOWED_ORIGINS	Según entorno	Origins CORS (ver settings)
DB_NAME	Sí	Nombre DB (default laqq_db)
DB_USER	Sí	Usuario Postgres
DB_PASSWORD	Sí	Password Postgres
DB_HOST	Sí en Docker	En compose: db
DB_PORT	Sí	En red Docker: 5432
LOAD_SEED_DATA	Opcional	true ejecuta scripts/seed_data.py (dev compose lo fuerza)
ENABLE_API_DOCS	Opcional	Swagger/ReDoc (default False; staff si se habilita)
COMPOSE_PROJECT_NAME	Opcional	Prefijo de recursos Docker (ej. laqq)

3.2 Email / negocio

Variable	Propósito
EMAIL_BACKEND	Backend Django de mail (consola en dev)
EMAIL_HOST, EMAIL_PORT, EMAIL_USE_TLS	SMTP
EMAIL_HOST_USER, EMAIL_HOST_PASSWORD	Credenciales SMTP
DEFAULT_FROM_EMAIL, DEFAULT_FROM_NAME	Remitente
BUSINESS_EMAIL	Destinatario consultas/contactos/tickets
QUOTES_EMAIL	Aviso cotizaciones (fallback BUSINESS_EMAIL)
RESEND_API_KEY	API Resend (si se usa)
BUSINESS_NAME, BUSINESS_PHONE, BUSINESS_ADDRESS	Datos en emails/UI
QUOTE_RESPONSE_TIME	Texto SLA en emails de cotización
FRONTEND_BASE_URL	Links absolutos en emails
BUSINESS_LOGO_PATH, BUSINESS_LOGO_URL	Logo en plantillas

3.3 Integraciones HTTP / Turnstile

Variable	Propósito
INTEGRATION_HTTP_TIMEOUT	Timeout cliente HTTP (default 15s)
INTEGRATION_HTTP_RETRIES	Reintentos (default 2)
PRODUCT_IMAGE_MAX_BYTES	Tope descarga imagen (default 5 MB)
PRODUCT_IMAGE_HOST_ALLOWLIST	Hosts CSV permitidos; vacío = cualquier http/https
ENABLE_PRODUCT_IMAGE_DOWNLOADS	On/off descargas en bulk
TURNSTILE_SECRET_KEY	Secret Cloudflare (backend)

3.4 Frontend (runtime / build)

Variable	Propósito
`FRONTEND_PORT`	Puerto publicado en **dev** (default 3000)
`VITE_API_BASE_URL`	Base API en **dev** (ej. `http://127.0.0.1:8000`). En **prod** el compose fija `/api` en build
`VITE_TURNSTILE_SITE_KEY`	Site key pública Turnstile
`VITE_SITE_ORIGIN`	Origin SEO (canonical/OG); prod default `https://laqq.com.ar`
`NGINX_CONFIG`	Archivo nginx en imagen FE (`nginx.conf` vs `nginx.dev.conf`)

---

4. Dependencias de despliegue

4.1 Imágenes y servicios

Pieza	Definición
Postgres	`postgres:15-alpine`
Backend prod	`Backend/Dockerfile.prod` → Gunicorn
Frontend prod	`Frontend/Dockerfile.prod` → build Vite + Nginx
Red	`laqq-prod` (bridge)

4.2 Volúmenes

Volumen	Uso
`db_data`	Datos PostgreSQL
`backend_static`	`collectstatic` → también montado en Nginx como `/var/www/static`
`backend_media`	Media uploads → Nginx `/var/www/media`

4.3 Let’s Encrypt / Certbot (prod)

En `docker-compose.prod.yml` el frontend monta:

- `/var/www/certbot:/var/www/certbot:ro`
- `/etc/letsencrypt:/etc/letsencrypt:ro`

Nginx (`Frontend/nginx.conf`) sirve el challenge ACME en HTTP y termina TLS con certificados bajo `/etc/letsencrypt/live/laqq.com.ar/`.
Los certificados deben existir **en el host** antes o durante el alta de HTTPS; el compose no genera el certificado por sí solo.

4.4 Proxy Nginx (resumen)

- HTTP → redirect HTTPS (canónico `laqq.com.ar`)

- `/api/` → backend Django (strip de `/api`)
- SPA + `/static` + `/media`

---

5. Migraciones, seeds y static

Al arrancar el contenedor backend, `Backend/entrypoint.sh` (prod) / `entrypoint.dev.sh` (dev):

1. Espera Postgres (`pg_isready`)
2. `python manage.py migrate --noinput`
3. Populate idempotente de datos de referencia:
    - `populate_user_data`
    - `populate_ticket_data`
    - `populate_contact_data`
    - `populate_quote_data`
4. Si `LOAD_SEED_DATA=true` → `python scripts/seed_data.py`
5. Asegura superuser de bootstrap si no existe (`username=laqq`, email `laqq@gmail.com`) — **cambiar en prod**
6. Si `DJANGO_ENV=production` → `collectstatic --noinput`

Populate manual (DB ya existente):

```
docker compose -f docker-compose.dev.yml exec backend python manage.py populate_user_da
docker compose -f docker-compose.dev.yml exec backend python manage.py populate_ticket_
docker compose -f docker-compose.dev.yml exec backend python manage.py populate_contact
docker compose -f docker-compose.dev.yml exec backend python manage.py populate_quote_d
```

Más detalle de seeds de producción: `docs/SEED_PRODUCCION.md`. Setup local ampliado: `docs/DEV_SETUP.md`.

---

6. CI/CD

Archivo: `.github/workflows/deploy.yml`

Triggers

- Push a `main` o `dev`
- Pull request hacia `dev`
- `workflow_dispatch`

Job `ci`

- Tests backend: `python manage.py test tickets contacts attachments quotes`
- `npm ci` + `npm run build` en Frontend

Job `deploy` (tras CI OK)

SSH al droplet y:

1. Clone o `git fetch` + `reset --hard origin/$BRANCH`
2. `docker compose -f docker-compose.prod.yml` → DB → build → backend (espera healthy) → frontend

Secrets usados (nombres; no valores)

Secret	Uso
`SSH_PRIVATE_KEY`	Deploy desde `main`
`SSH_PRIVATE_KEY_DEV`	Deploy no-main / PR
`DROPLET_HOST`	Host prod
`DROPLET_HOST_DEV`	Host dev
`DEPLOY_DIR`	Directorio remoto del repo

El script remoto usa usuario SSH `deploy@…`. El README menciona también `DROPLET_USER`; el workflow actual fija `deploy@` en el comando `ssh`.

---

7. Checklist post-deploy

- [ ] `docker compose -f docker-compose.prod.yml ps` — servicios healthy
- [ ] HTTP/HTTPS: home carga (`curl -I https://<dominio>/`)
- [ ] API vía proxy: `GET https://<dominio>/api/` (api root) o health admin login
- [ ] Login backoffice (`/login`) con usuario admin
- [ ] Media/static: una imagen de producto o static de admin carga
- [ ] Cotización pública: Turnstile + email (o log) OK
- [ ] `DEBUG=False`, `SECRET_KEY` no default, `ALLOWED_HOSTS` incluye el dominio
- [ ] Superuser bootstrap cambiado o deshabilitado según política de seguridad
- [ ] Certificados TLS válidos y renovación Certbot operativa en el host

---

8. Troubleshooting breve

Síntoma	Qué revisar
DB no conecta	`DB_HOST=db` dentro de compose; healthcheck Postgres; logs `laqq-db`
`DisallowedHost`	Agregar dominio / `backend` a `ALLOWED_HOSTS`
CORS / CSRF fallan	`CORS_ALLOWED_ORIGINS`, `CSRF_TRUSTED_ORIGINS` con scheme correcto (`https://…`)
Front llama mal a la API	Dev: `VITE_API_BASE_URL`. Prod: rebuild con `/api` y Nginx

404/502 en `/api`	Backend unhealthy; proxy Nginx; logs `laqq-backend` / `laqq-frontend`
Permisos media	Entrypoint hace `chown` de `mediafiles`/`staticfiles`; volumen host vs container
Cotizaciones/tickets fallan por estados	Correr populate_* (datos de referencia)
SMTP 535	En dev usar backend consola; en prod app password / Resend
Puerto ocupado	Cambiar mapeo o `FRONTEND_PORT`

Referencias

- `docker-compose.dev.yml`, `docker-compose.prod.yml`
- `scripts/dev-up.sh`, `scripts/prod-up.sh`, `scripts/dev-down.sh`, `scripts/prod-down.sh`
- `Backend/entrypoint.sh`, `Backend/Dockerfile.prod`
- `Frontend/Dockerfile.prod`, `Frontend/nginx.conf`
- `env.example`
- `.github/workflows/deploy.yml`
- Arquitectura: `docs/MANUAL_ARQUITECTURA.md`